

Penetration Testing Lab Report

Web Application OS Command Injection Vulnerability

Project Title: Command Injection Investigation and Flag Extraction

Target Application: StaffHub (localhost:8080)

Author : Sudip Bastola

Assessment Date: August 13, 2026

Assessment Type: Controlled Academic / Lab Web Application Security Assessment

1. Vulnerability Information

- **Vulnerability Name:** OS Command Injection
- **CWE ID:** CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
- **OWASP Category:** OWASP Top 10:2021 - A03: Injection
- **Severity:** High
- **Justification:** Allowed an authenticated portal user to execute arbitrary operating-system commands directly on the hosting server under the staffhub service user context, enabling directory enumeration, filesystem searching, and unauthorized reading of internal flag files.
- **Location:** <http://localhost:8080/tools.php>
- **Vulnerable Parameter:** IP Address input field (IT Support Tools - Employee Workstation Connectivity Test)

2. Description

OS Command Injection is a web application vulnerability that occurs when user-supplied input is directly incorporated into an operating-system command without adequate server-side validation, sanitization, or escaping. Because the backend environment fails to separate user input from executable instructions, the system shell interprets the input as shell commands rather than plain text data.

During the assessment of the StaffHub web application, the IT Support Tools functionality on tools.php was tested. This feature accepts an IP address to test network connectivity to employee workstations using a backend ping utility. By appending a shell command separator (;) followed by additional commands, the backend executed the intended ping operation and then sequentially ran the injected system commands. In testing, the application executed commands including ls, find, and cat, which allowed for directory listing, full filesystem search, and direct reading of server-side files. This vulnerability is dangerous because an attacker can execute arbitrary operating-system commands, inspect internal configuration files, access sensitive stored data, and potentially compromise the underlying hosting server.

3. Tools Used

- **Brave Browser:** Used to access the StaffHub web application, interact with the IT Support Tools page, submit baseline IP address values and command injection payloads, and observe terminal output returned by the application.

4. Step-by-Step Exploitation

Step 1 : Testing the Provided IP Addresses

- **Action:** Navigated to `http://localhost:8080/tools.php` and tested each IP address provided in the Quick Reference section.
- **Reason:** To establish the normal behavior of the connectivity testing functionality using the IP addresses supplied by the application.
- **Command:** 192.168.1.10, 192.168.1.20, 192.168.1.30, 192.168.1.50
- **Observation:** The tested IP addresses did not return a successful connection response.
- **Security Significance:** This established the initial baseline behavior of the application before testing an alternative local address.

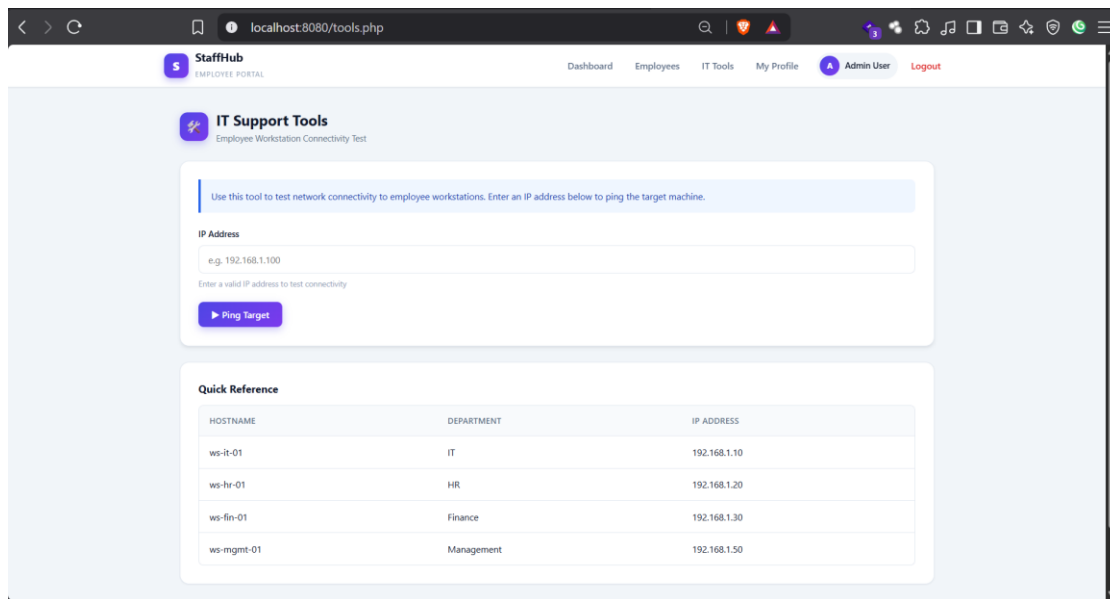


Figure 1 StaffHub IT Support Tools Showing Initial IP Address Testing

Step 2 :Testing Localhost

- **Action:** Entered 127.0.0.1 into the IP Address input field on tools.php and clicked the Ping Target button
- **Reason:** To determine whether the connectivity test would successfully communicate with the local system and to establish a working baseline for further input testing.
- **Command:** 127.0.0.1
- **Explanation:** The address 127.0.0.1 refers to the localhost interface of the system. Testing this value provided a known local target for confirming that the backend connectivity function was executing successfully.
- **Security Significance:** The successful localhost response confirmed that the input was being processed by the application's connectivity-testing functionality, providing a working baseline before testing whether the input could be manipulated.
- **Observation:** The application successfully returned a connection response. The ping output showed 4 packets transmitted, 4 packets received, and 0% packet loss.

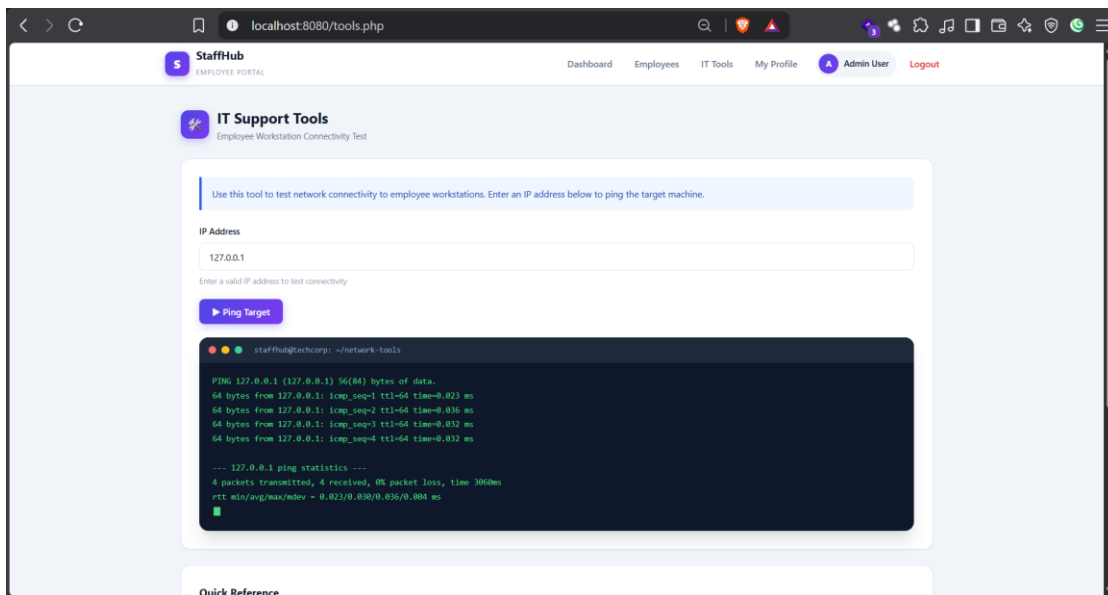


Figure 2 Successful Ping of 127.0.0.1

Step 3 :Testing for Command Injection

- **Action:** Navigated to <http://localhost:8080/tools.php> and entered 127.0.0.1 ;ls into the IP Address input field, then clicked the Ping Target button.
- **Command:** 127.0.0.1 ;ls
- **Reason:** In Unix-like shell environments, the semicolon (;) serves as a command separator that allows multiple distinct commands to run sequentially. This was tested to determine whether the application's backend ping function concatenated raw user input directly into a shell execution string without stripping shell metacharacters.
- **Explanation:** Breakdown of the payload components:
 - **127.0.0.1 :** The baseline IP address passed to satisfy the initial ping command.
 - **; :** The shell command separator instructing the interpreter to execute a subsequent command after the ping finishes.
 - **ls :** The standard Unix command used to list directory contents in the current working directory.
- **Observation/Result:** The application successfully executed the ping command against 127.0.0.1 and immediately followed it by executing ls. The terminal display output revealed the files and directories inside the web root.
- **Security Significance:** This result confirms that the input parameter is passed directly to an underlying operating system shell, demonstrating that arbitrary shell commands can be executed beyond the intended network connectivity test.

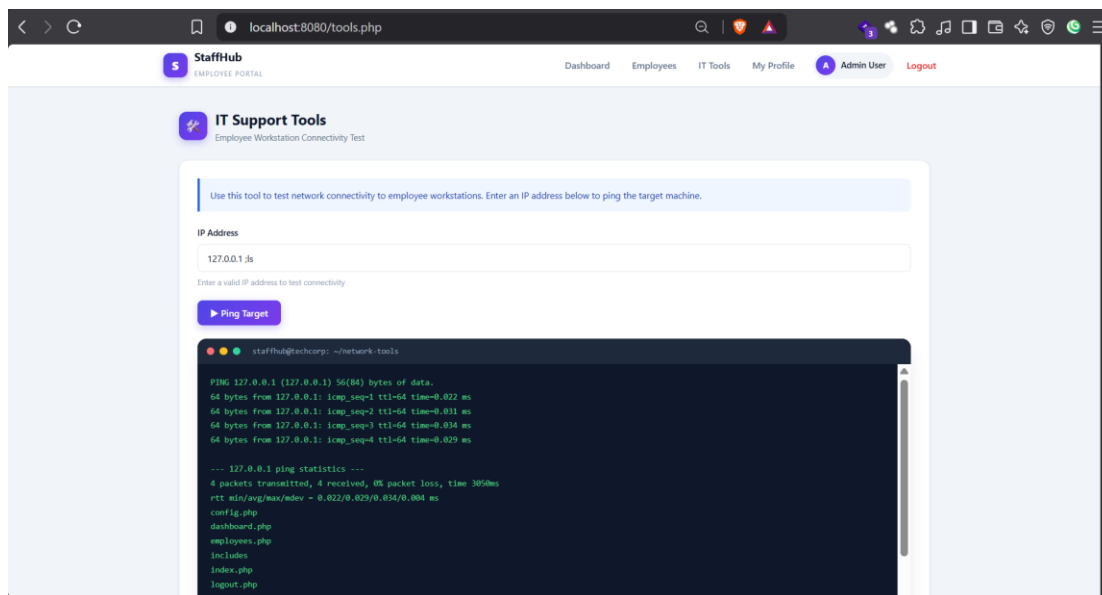


Figure 3 Command Injection Using the 127.0.0.1 ;ls Payload

Step 4 : Filesystem Enumeration

- **Action:** Submitted a search command using the find utility into the **IP Address** field on tools.php.
- **Command:** 127.0.0.1; find / -iname "**flag**"
- **Reason:** To determine whether the command injection flaw could be used to traverse and enumerate the wider server filesystem to locate specific flag files.
- **Explanation:** Breakdown of the command components:
 - **127.0.0.1** : The baseline target address for the ping command.
 - **;** : The shell command separator.
 - **find** : The system utility used to search for files and directories.
 - **/** : The starting root directory path for the search.
 - **-iname** : Performs a case-insensitive search matching the specified pattern.
 - ****flag**** : Search pattern targeting any filename containing the string "flag".

- **Observation/Result:** The application executed the ping command and returned the output of the find operation, displaying matching paths across the filesystem.
- **Security Significance:** Demonstrates that the web server process has broad read access across the local filesystem, enabling directory mapping and identifying target files for extraction.

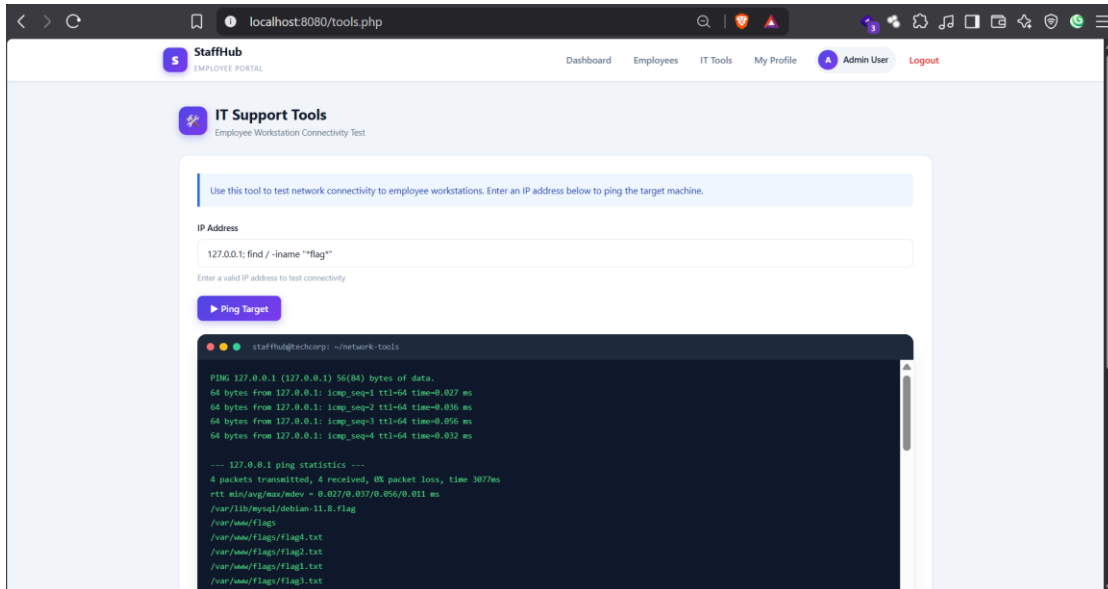


Figure 4 Filesystem Enumeration Using the find Command

Step 5 : Reading Flag 3

- **Action:** Entered a command to read the contents of the identified `/var/www/flags/flag3.txt` file into the **IP Address** field.
- **Command:** `127.0.0.1; cat /var/www/flags/flag3.txt`
- **Reason:** To verify whether command injection could be leveraged to read sensitive, restricted files directly from the filesystem.
- **Explanation:** Breakdown of the command components:
 - **127.0.0.1** :The baseline target address for the ping command.
 - **;** : The shell command separator.
 - **Cat** : The standard Unix utility used to concatenate and display the contents of a specified file.
 - **/var/www/flags/flag3.txt** : The absolute path to the target flag file identified in Step 2.
- **Observation/Result:** Following the standard ping statistics output, the embedded terminal displayed the cleartext contents of the file.
- **Security Significance:** Successfully reading the contents of `flag3.txt` confirms arbitrary file disclosure and fully validates the OS Command Injection vulnerability.

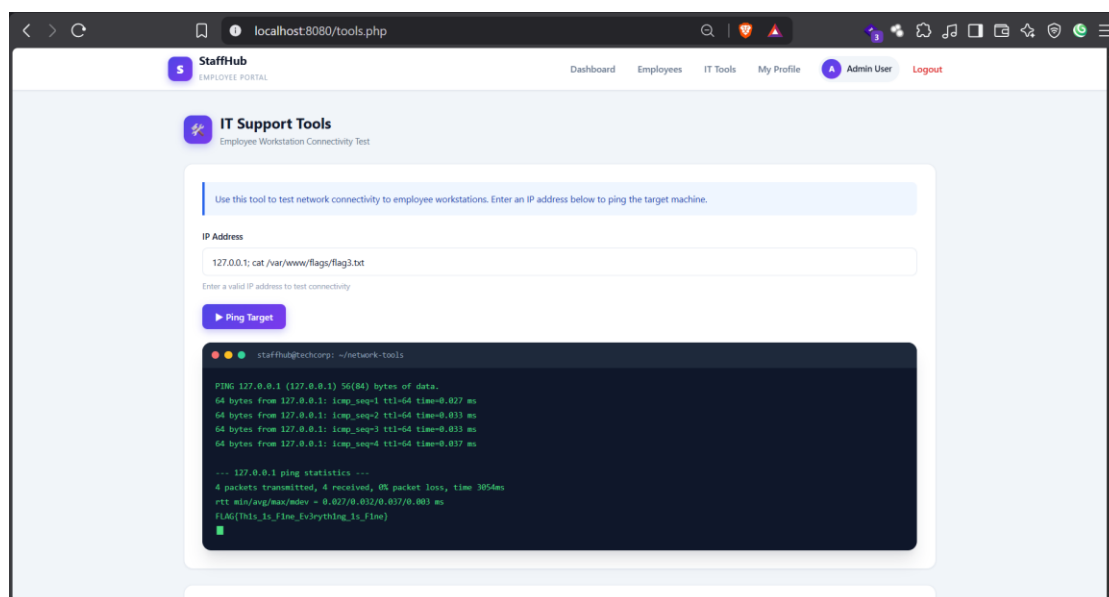


Figure 5 Flag 3 Retrieved Through Command Injection

5. Proof of Exploitation

The evidence below confirms that the IP Address input field on tools.php allows arbitrary command execution, enabling directory enumeration, filesystem searching, and direct file extraction, culminating in the successful retrieval of Flag 3.

- **Exact Flag:** FLAG{Th1s_1s_F1ne_Ev3ryth1ng_1s_F1ne}
- **Date:** August 14, 2026

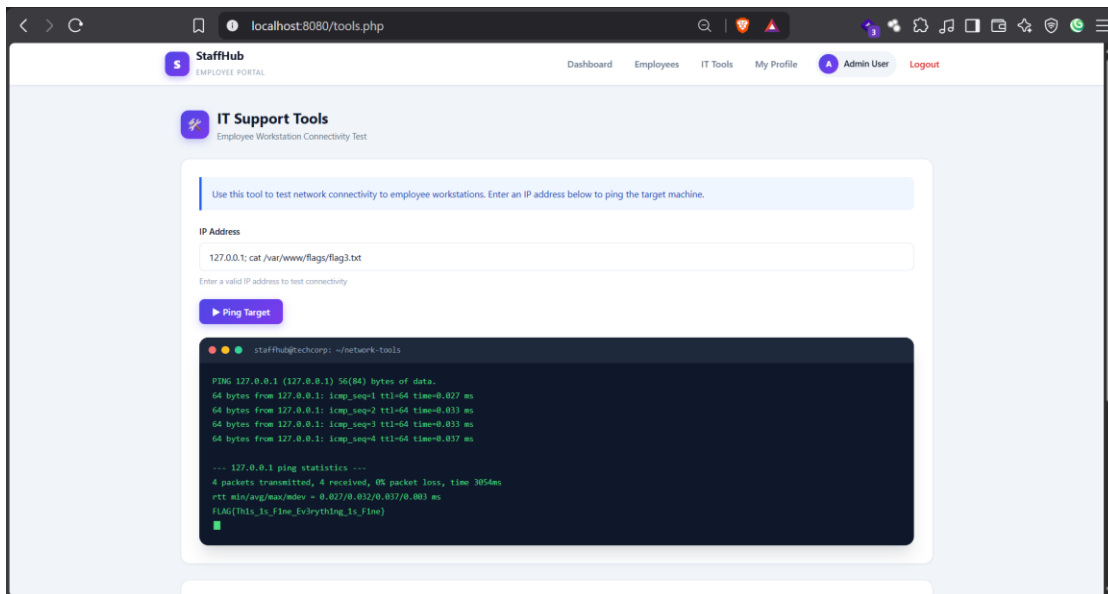


Figure 6 Flag 3 Displayed in Terminal Output After Successful OS Command Injection

6. Business Impact

Demonstrated in the Lab

During testing on the StaffHub application, submitting shell metacharacters through the IP Address parameter on tools.php allowed the execution of arbitrary system commands. I demonstrated sequential execution of ls to list web application files, find to enumerate files across the server filesystem, and cat to read the cleartext contents of /var/www/flags/flag3.txt.

Potential Real-World Impact

In a production enterprise deployment, an unmitigated OS Command Injection vulnerability presents severe operational and business risks. An attacker could read application source code and configuration files containing sensitive database passwords or API keys. With the ability to execute system binaries, a malicious user could establish an interactive reverse shell, modify or delete critical application files, pivot laterally into internal network segments, or deploy unauthorized payloads. Such compromises can result in extensive data breaches, prolonged service disruption, compliance violations, and severe reputational damage.

7. Remediation Recommendation

To resolve this OS Command Injection vulnerability, StaffHub should implement the following server-side security controls:

- **Strict Server-Side IP Validation (Allowlisting):** Implement strict server-side format validation to verify that the submitted value is a legitimate IPv4 address before invoking any backend processing
- **Avoid Shell-Based Command Construction:** Avoid concatenating user input directly into shell execution strings. If an external process must be executed, use safe process execution functions.
- **Use Native Networking Functions:** Rather than invoking operating-system shell utilities like ping, perform connectivity checks, eliminating OS-level process execution entirely.
- **Principle of Least Privilege:** Run the web server and its associated processes under a dedicated service account with minimal filesystem and operating-system privileges. Restrict read access to sensitive system directories and ensure the web service account cannot execute unauthorized administrative binaries.
- **Security Logging and Monitoring:** Implement centralized server-side logging to detect and alert on anomalous input submissions containing command separators, unusual subshell invocations, or unexpected child process spawning from the web server daemon.

References

Common Weakness Enumeration, n.d. *CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection*. [Online]

Available at: <https://cwe.mitre.org/data/definitions/78.html>

[Accessed 13 08 2026].

OWASP Cheat Sheet Series, n.d. *OS Command Injection Defense Cheat Sheet*. [Online]

Available at:

https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html#introduction

[Accessed 13 08 2026].

OWASP Top 10:2021, n.d. *A03:2021 – Injection*. [Online]

Available at: https://owasp.org/Top10/2021/A03_2021-Injection/

[Accessed 13 08 2026].

OWASP, n.d. *Command Injection*. [Online]

Available at: https://owasp.org/www-community/attacks/Command_Injection

[Accessed 13 08 2026].

php, n.d. *filter_var*. [Online]

Available at: <https://www.php.net/manual/en/function.filter-var.php>

[Accessed 13 08 2026].